

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)*

Assessment Tool Weightage: 50%

Dr G.A. SENTHIL

QUESTIONS: 10

Total Marks:50

Annexure A

Constraint - Based Problem-Solving

Code of Conduct:

I Vignesh S (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Vignesh S

Qn 1)

```
import numpy as np
import random

grid_size = 5
goal = (4, 4)
obstacles = [(2, 2), (1, 3)]

actions = [(0,1),(0,-1),(1,0),(-1,0)]
Q = np.zeros((grid_size, grid_size, 4))

alpha = 0.1
gamma = 0.9
epsilon = 0.2

for episode in range(500):
    state = (0,0)

    while state != goal:
        if random.random() < epsilon:
            action = random.randint(0,3)
        else:
            action = np.argmax(Q[state[0], state[1]])

        move = actions[action]
        next_state = (
            max(0,min(grid_size-1,state[0]+move[0])),
            max(0,min(grid_size-1,state[1]+move[1]))
        )

        if next_state in obstacles:
            reward = -10
            next_state = state
        elif next_state == goal:
            reward = 100
        else:
            reward = -1

        Q[state[0],state[1],action] += alpha * (
            reward + gamma * np.max(Q[next_state[0],next_state[1]])
            - Q[state[0],state[1],action]
        )

        state = next_state

print(Q)
```

OUTPUT

```
[[[ 42.612659  35.50753468  35.65804823  33.29800512]
 [ 21.01362423  34.63906674  48.45851  41.03259754]
 [ -1.59407816  36.40362354 -1.50219043  4.18067 ]
 [ -0.85789868 -1.07008983 -2.85121016 -1.12801184]
 [ -0.57655314 -0.65958253  1.90429964 -0.67092743]]

[[ 48.03295396  2.50494297  5.10658019  10.51269851]
 [ 25.28090448  36.0772417  54.9539  41.00524136]
 [ -2.89499136  41.92302875 -0.70436753  1.56446935]
 [ 0.  0.  0.  0. ]
 [ -0.199 -2.58223474  12.60692871 -0.199 ]]]

[[ 9.48333827  2.70002839  47.44625895  2.95158267]
 [ 39.88704038  29.4651532  62.171  45.26041385]
 [ 0.  0.  0.  0. ]
 [ 7.48667431 -1. -0.1 -1. ]
 [ 2.56116203 -0.19  44.89610177  0.3801625 ]]]

[[ 61.75409308  1.78713499  1.52888644  3.82457564]
 [ 70.19  47.48189858  61.17332324  50.98302689]
 [ 68.7363611  58.91108588  79.1  57.82226088]
 [ 23.6919227  7.83350974  88.47030407  0.3029835 ]
 [ -0.1 -0.1  87.84233454  0. ]]]

[[ 35.96680467 -0.48930562  0.81934072 -0.37246036]
 [ 78.69646011  7.24441518  4.44133739  11.47625548]
 [ 89.  58.82193092  76.83509274  64.15437227]
 [ 100.  75.90683828  84.90833709  71.29058079]
 [ 0.  0.  0.  0. ]]]]
```

Qn 2)

```
import numpy as np
import random
```

```
arms = [0.2, 0.5, 0.8]
epsilons = [0.1, 0.3, 0.5]
```

```
for epsilon in epsilons:
```

```
    Q = np.zeros(3)
```

```
    N = np.zeros(3)
```

```
    reward_sum = 0
```

```
    for i in range(500):
```

```
        if random.random() < epsilon:
```

```
            arm = random.randint(0,2)
```

```
        else:
```

```
            arm = np.argmax(Q)
```

```
        reward = 1 if random.random() < arms[arm] else 0
```

```
        N[arm] += 1
```

```
        Q[arm] += (reward - Q[arm]) / N[arm]
```

```
        reward_sum += reward
```

```
    print("Epsilon:", epsilon)
```

```
    print("Rewards:", reward_sum)
```

OUTPUT:

```
Epsilon: 0.1
Rewards: 380
Epsilon: 0.3
Rewards: 359
Epsilon: 0.5
Rewards: 336
```

Qn 3)

```
states = ["Start", "Road", "Destination"]
actions = ["Move", "Wait"]
```

```
transition = {
    "Start": {"Move": "Road"},
    "Road": {"Move": "Destination"},
    "Destination": {"Move": "Destination"}
}
```

```
reward = {
    "Start": -2,
    "Road": -1,
    "Destination": 100
}
```

```
energy = 20
```

```
state = "Start"
```

```
while state != "Destination" and energy > 0:
    action = "Move"
    state = transition[state][action]
    energy -= 5
    print(state, reward[state], energy)
```

OUTPUT:

```
Road -1 15
Destination 100 10
```

Qn 4)

```
import numpy as np
```

```
grid = 4
gamma = 0.9
```

```
V = np.zeros((grid,grid))
```

```
goal = (3,3)
```

```
for iteration in range(100):
    newV = V.copy()
```

```
    for i in range(grid):
        for j in range(grid):
```

```
            if (i,j)==goal:
                continue
```

```

values=[]

for dx,dy in [(1,0),(-1,0),(0,1),(0,-1)]:

    ni=max(0,min(grid-1,i+dx))
    nj=max(0,min(grid-1,j+dy))

    reward=-1

    values.append(reward+gamma*V[ni,nj])

newV[i,j]=max(values)

V=newV

print(V)

```

OUTPUT:

```

[[ -4.68559  -4.0951  -3.439   -2.71   ]
 [ -4.0951  -3.439   -2.71   -1.9    ]
 [ -3.439   -2.71   -1.9    -1.     ]
 [ -2.71    -1.9    -1.     0.     ]]

```

Qn 5)

```

battery = 30
deliveries = 0

while battery >= 5:
    deliveries += 1
    battery -= 5
    print("Delivery", deliveries)

print("Maximum Deliveries =", deliveries)

```

OUTPUT:

```

Delivery 1
Delivery 2
Delivery 3
Delivery 4
Delivery 5
Delivery 6
Maximum Deliveries = 6

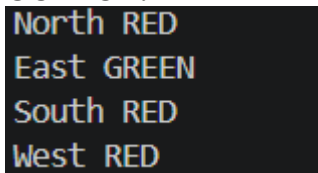
```

Qn 6)

```
signals = ["North", "East", "South", "West"]
```

```
emergency = "East"
```

```
for signal in signals:
    if signal == emergency:
        print(signal, "GREEN")
    else:
        print(signal, "RED")
```

OUTPUT:

```
North RED
East GREEN
South RED
West RED
```

Qn 7)

```
import random
```

```
bandwidth = 100
```

```
for packet in range(10):
```

```
    usage = random.randint(5,20)
```

```
    if bandwidth >= usage:
        bandwidth -= usage
        reward = 10
        print("Packet Sent", reward)
    else:
        reward = -5
        print("Dropped", reward)
```

```
print("Remaining Bandwidth:", bandwidth)
```

OUTPUT:

```
Packet Sent 10
Packet Sent 10
Packet Sent 10
Packet Sent 10
Packet Sent 10
Packet Sent 10
Packet Sent 10
Packet Sent 10
Dropped -5
Packet Sent 10
Remaining Bandwidth: 5
```

Qn 8)

```
import random

arms = [0.3,0.6,0.8]

random_reward = 0

for i in range(500):
    arm = random.randint(0,2)
    random_reward += 1 if random.random()<arms[arm] else 0

print("Random Reward:", random_reward)

epsilon = 0.1

Q=[0,0,0]
N=[0,0,0]
reward_sum=0

for i in range(500):

    if random.random()<epsilon:
        arm=random.randint(0,2)
    else:
        arm=Q.index(max(Q))

    reward=1 if random.random()<arms[arm] else 0

    N[arm]+=1
    Q[arm]+=(reward-Q[arm])/N[arm]

    reward_sum+=reward

print("Epsilon Greedy Reward:", reward_sum)
```

OUTPUT:

```
Random Reward: 272
Epsilon Greedy Reward: 373
```

Qn 9)

```
battery = 100

no_fly = [3,5]

for location in range(1,8):

    if location in no_fly:
        print("Avoid Location", location)
        continue

    if battery < 15:
        print("Battery Low")
        break

    battery -= 15

    print("Delivered at", location)

print("Battery Left:", battery)
```

OUTPUT:

```
Delivered at 1
Delivered at 2
Avoid Location 3
Delivered at 4
Avoid Location 5
Delivered at 6
Delivered at 7
Battery Left: 25
```

Qn 10)

```
import random

threshold = 80

usage = 0

for hour in range(24):

    consume = random.randint(2,6)
```

```
if usage + consume <= threshold:
    usage += consume
    reward = 10
else:
    reward = -10

print("Hour:", hour+1, "Usage:", usage, "Reward:", reward)

print("Total Energy Used:", usage)
```

OUTPUT:

```
Hour: 1 Usage: 4 Reward: 10
Hour: 2 Usage: 9 Reward: 10
Hour: 3 Usage: 14 Reward: 10
Hour: 4 Usage: 19 Reward: 10
Hour: 5 Usage: 24 Reward: 10
Hour: 6 Usage: 28 Reward: 10
Hour: 7 Usage: 30 Reward: 10
Hour: 8 Usage: 35 Reward: 10
Hour: 9 Usage: 40 Reward: 10
Hour: 10 Usage: 45 Reward: 10
Hour: 11 Usage: 48 Reward: 10
Hour: 12 Usage: 51 Reward: 10
Hour: 13 Usage: 54 Reward: 10
Hour: 14 Usage: 57 Reward: 10
Hour: 15 Usage: 63 Reward: 10
Hour: 16 Usage: 68 Reward: 10
Hour: 17 Usage: 73 Reward: 10
Hour: 18 Usage: 77 Reward: 10
Hour: 19 Usage: 79 Reward: 10
Hour: 20 Usage: 79 Reward: -10
Hour: 21 Usage: 79 Reward: -10
Hour: 22 Usage: 79 Reward: -10
Hour: 23 Usage: 79 Reward: -10
Hour: 24 Usage: 79 Reward: -10
Total Energy Used: 79
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided